ЛАБОРАТОРНА РОБОТА № 1
Дослідження ефективності послідовної та паралельної обробки даних у Python

Мета роботи. Метою лабораторної роботи є набуття практичних навичок організації паралельної обробки даних у Python, порівняння послідовного та паралельного способів виконання обчислень, а також експериментальне дослідження впливу кількості паралельних виконавців на продуктивність програми.

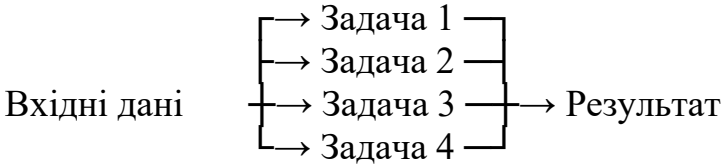
Теоретичні відомості

Паралельна обробка даних – це спосіб організації обчислень, за якого окремі частини задачі виконуються одночасно або з перекриттям у часі.

У послідовному підході задачі виконуються одна за одною:

Задача 1 → Задача 2 → Задача 3 → Задача 4

У паралельному підході незалежні частини задачі можуть виконуватися одночасно:



Основна ідея полягає в тому, щоб розподілити загальну роботу між декількома виконавцями.

Якщо задача є достатньо великою та може бути розділена на незалежні частини, її виконання можна прискорити шляхом використання декількох обчислювальних ресурсів.

Для реалізації послідовних т апаралельної обробки даних можна використовувати Python.

Одними з основних засобів є:

```
import random
import time
import os
```

а для організації паралельного виконання:

```
from concurrent.futures import ProcessPoolExecutor
```

Хід роботи та завдання до лабораторної роботи

Формування вхідного набору даних. Необхідно сформувати масив випадкових чисел. Наприклад:

```
import random

data = [random.randint(1, 1_000_000) for _ in range(1_000_000)]
```

Однак для проведення коректного експерименту бажано зафіксувати генератор випадкових чисел:

```
random.seed(42)
```

Тоді при кожному запуску програми буде формуватися один і той самий масив.

Це дозволяє коректно порівнювати: послідовний алгоритм; паралельний алгоритм; різну кількість процесів.

Необхідно виконати обчислювальну обробку кожного елемента масиву. Для кожного числа *x* необхідно виконати задану математичну функцію:

$$f(x) = \sum_{i=1}^{K} \left(\sqrt{x+i} \cdot \sin(x+i) \right)$$

де *K* – параметр, що задається відповідно до варіанта.

Отримані значення необхідно використати для формування нового масиву.

Приклад функції Python:

```
import math

def process_value(x, k):
    result = 0

    for i in range(1, k + 1):
        result += math.sqrt(x + i) * math.sin(x + i)

    return result
```

Завдання 1. Генерація даних. Створити масив випадкових цілих чисел. Розмір масиву та діапазон значень визначити за номером варіанта.

Варіант	Кількість елементів	Діапазон	K
1	500 000	1–10 000	100
2		1–20 000	150
3		1–30 000	200
4		1–10 000	100
5		1–20 000	150
6		1–30 000	200
7		1–10 000	100
8		1–20 000	150
9		1–30 000	200
10		1–10 000	100
11		1–20 000	150
12		1–30 000	200
13		1–10 000	100
14		1–20 000	150

15		1–30 000	200
16		1–10 000	100
17		1–20 000	150
18		1–30 000	200
19		1–20 000	150
20		1–30 000	200

Завдання 2. Реалізація послідовної обробки

Реалізувати функцію, яка послідовно обробляє всі елементи масиву.

Наприклад:

```
def sequential_processing(data, k):
    result = []

    for x in data:
        result.append(process_value(x, k))

    return result
```

Необхідно перевірити правильність роботи функції на невеликому наборі даних.

Наприклад:

```
test_data = [10, 20, 30, 40, 50]
```

Завдання 3. Виміряти час послідовної обробки. Для вимірювання часу використати модуль `time`. Отримане значення необхідно зберегти.

Завдання 4. Розподіл даних.

Для паралельної обробки необхідно розділити початковий масив на декілька частин (пропонується 5).

Для цього можна використати функцію:

```
def split_data(data, parts):
    chunk_size = len(data) // parts

    chunks = []

    for i in range(parts):
        start = i * chunk_size

        if i == parts - 1:
            end = len(data)
        else:
            end = (i + 1) * chunk_size

        chunks.append(data[start:end])

    return chunks
```

Завдання 5. Реалізація паралельної обробки. Для організації паралельного виконання використати ProcessPoolExecutor.

Завдання 6. Дослідження різної кількості процесів. Рекомендується взяти 1, 2, 4, 8

Завдання 7. Вимірювання продуктивності. Розрахунок Speedup та Efficiency

Для кожної кількості процесів необхідно виміряти час виконання.

Результати занести до таблиці.

Кількість процесів	Час виконання, с	Speedup	Efficiency
1		1,00	100%
2			
4			
8			

Прискорення, Speedup, визначається як:

$$S_p = \frac{T_1}{T_p}$$

де:

T_1 – час послідовного виконання;

T_p – час виконання з використанням pp процесів;

S_p – прискорення.

Ефективність використання процесів визначається:

$$E_p = \frac{S_p}{p}$$

або у відсотках:

$$E_p = \frac{S_p}{p} \cdot 100\%$$

Завдання 8. Перевірка правильності результатів

Необхідно переконатися, що послідовна та паралельна реалізації дають однаковий результат.

Наприклад:

```
print(result_sequential == result_parallel)
```

Очікуваний результат:

```
True
```

Для чисел із плаваючою комою через особливості обчислень доцільніше використовувати порівняння з допустимою похибкою.

Наприклад:

```
import math
all(
    math.isclose(a, b, rel_tol=1e-9, abs_tol=1e-9)
    for a, b in zip(result_sequential, result_parallel)
)
```

Завдання 9. Побудова графіків

Побудувати щонайменше два графіки.

Графік 1.Залежність часу виконання від кількості процесів:

X – кількість процесів

Y – час виконання

Графік 2. Залежність Speedup від кількості процесів:

X – кількість процесів

Y – Speedup

Для другого графіка бажано додатково показати теоретичну лінію ідеального прискорення:

$$S_p = p$$

Контрольні питання

1. Що таке паралельна обробка даних?
2. Чим послідовна обробка відрізняється від паралельної?
3. Які задачі доцільно виконувати паралельно?
4. Що таке CPU-bound задача?
5. Що таке процес?
6. Чим процес відрізняється від потоку?
7. Для чого використовується `ProcessPoolExecutor`?
8. Що таке пул процесів?
9. Навіщо розділяти набір даних на частини?
10. Що таке Speedup?
11. Як розраховується Speedup?
12. Що таке Efficiency?
13. Як розраховується Efficiency?
14. Чому реальне прискорення зазвичай менше за теоретичне?
15. Чому збільшення кількості процесів не завжди приводить до прискорення?
16. Що таке накладні витрати паралельного виконання?
17. Який зв'язок між кількістю процесів і кількістю ядер CPU?
18. Чому необхідно перевіряти однаковість результатів послідовного та паралельного алгоритмів?
19. У яких випадках паралельна обробка може бути недоцільною?